Lecture – 8
Advanced Sorting Methods –Part 3

M.G.Noel A.S Fernando (PhD)

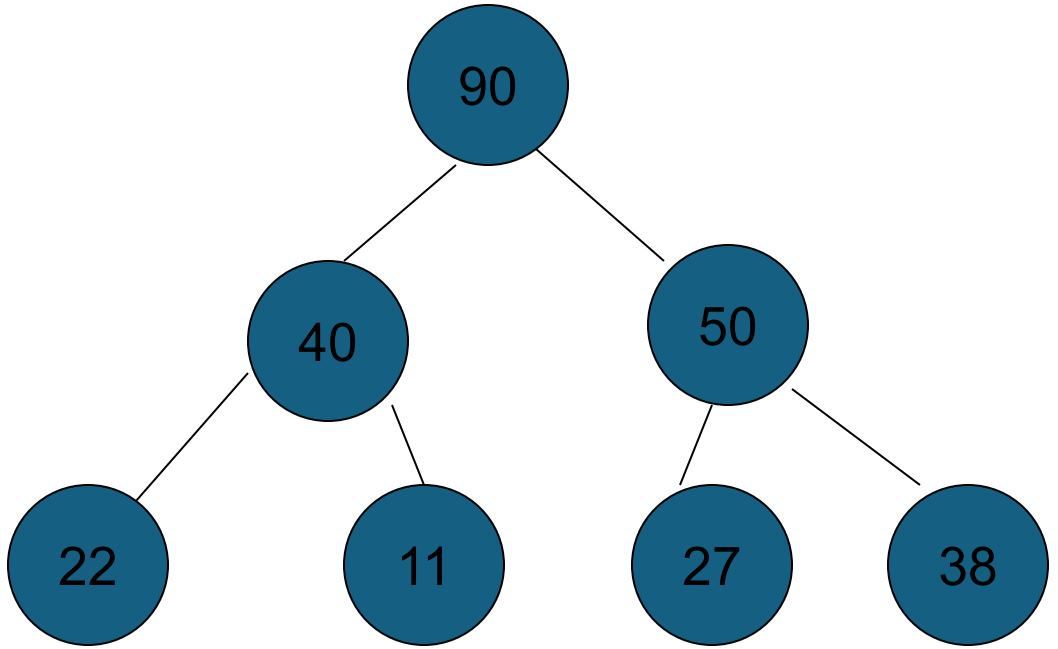
Professor

NSBM/Dept. of Information Systems Engineering, UCSC

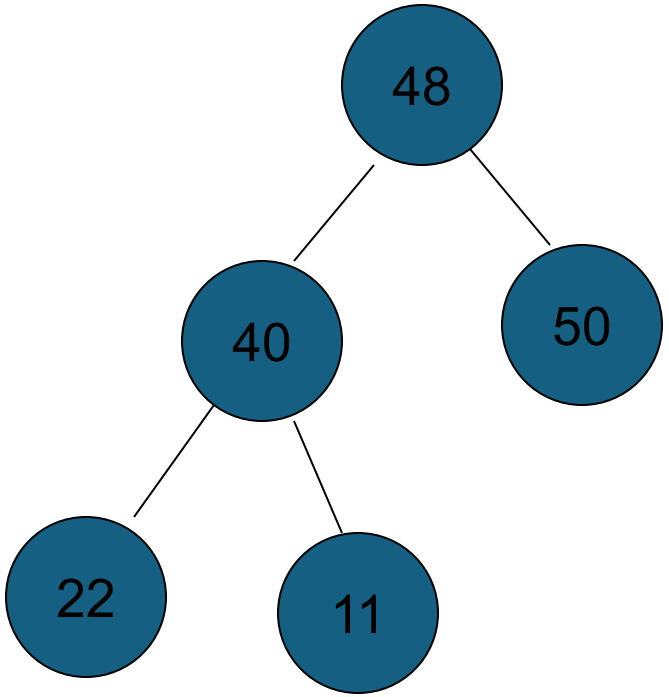
Heaps, Heap sorts and Priority Queues

- Heap
- A heap of size n is a B-tree of n nodes such that the values of each node is >= the value of its children. (if any)

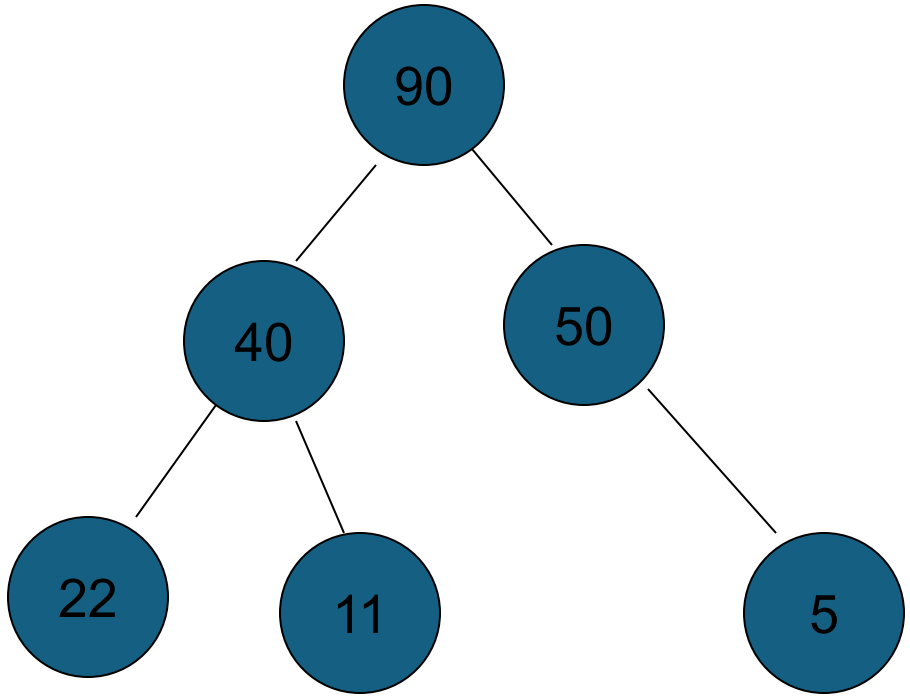
Examples



A Heap



Not a heap [ie 50 <48]



Not a Heap [Gaps]

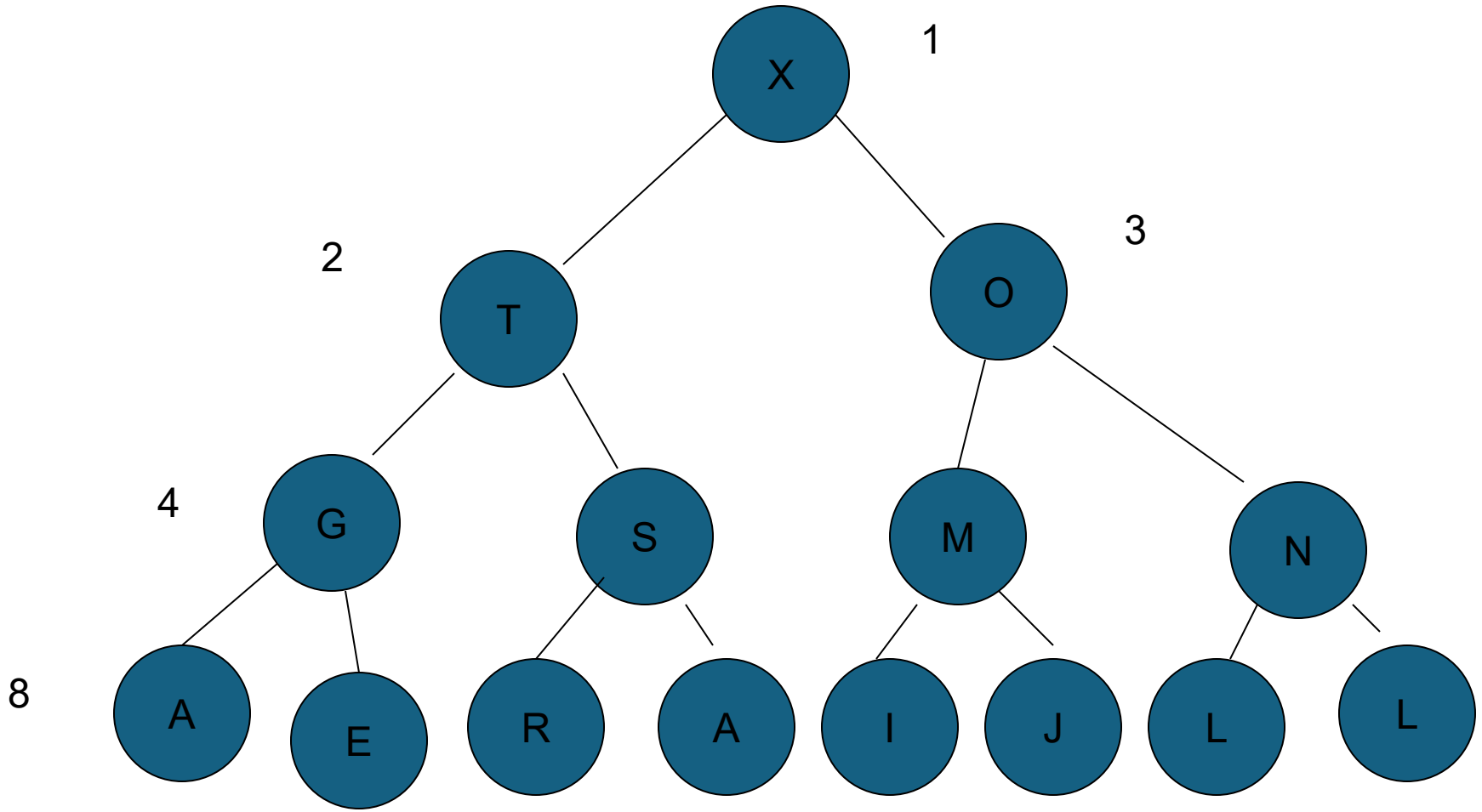
Properties

- A heap, however, is a special kind of binary tree that leaves no gaps in an array implementation.
- The definition of the heap data type is as follows.

A heap is a binary tree satisfying the following restrictions.

- All leaves are on adjacent levels.
- All leaves on the lowest level occur at the left of the tree
- All levels above the lowest are completely filled.
- Both children of any node are again heaps.
- The values stored at any node is at least as large as the values in its two children.

Array representation of a heap

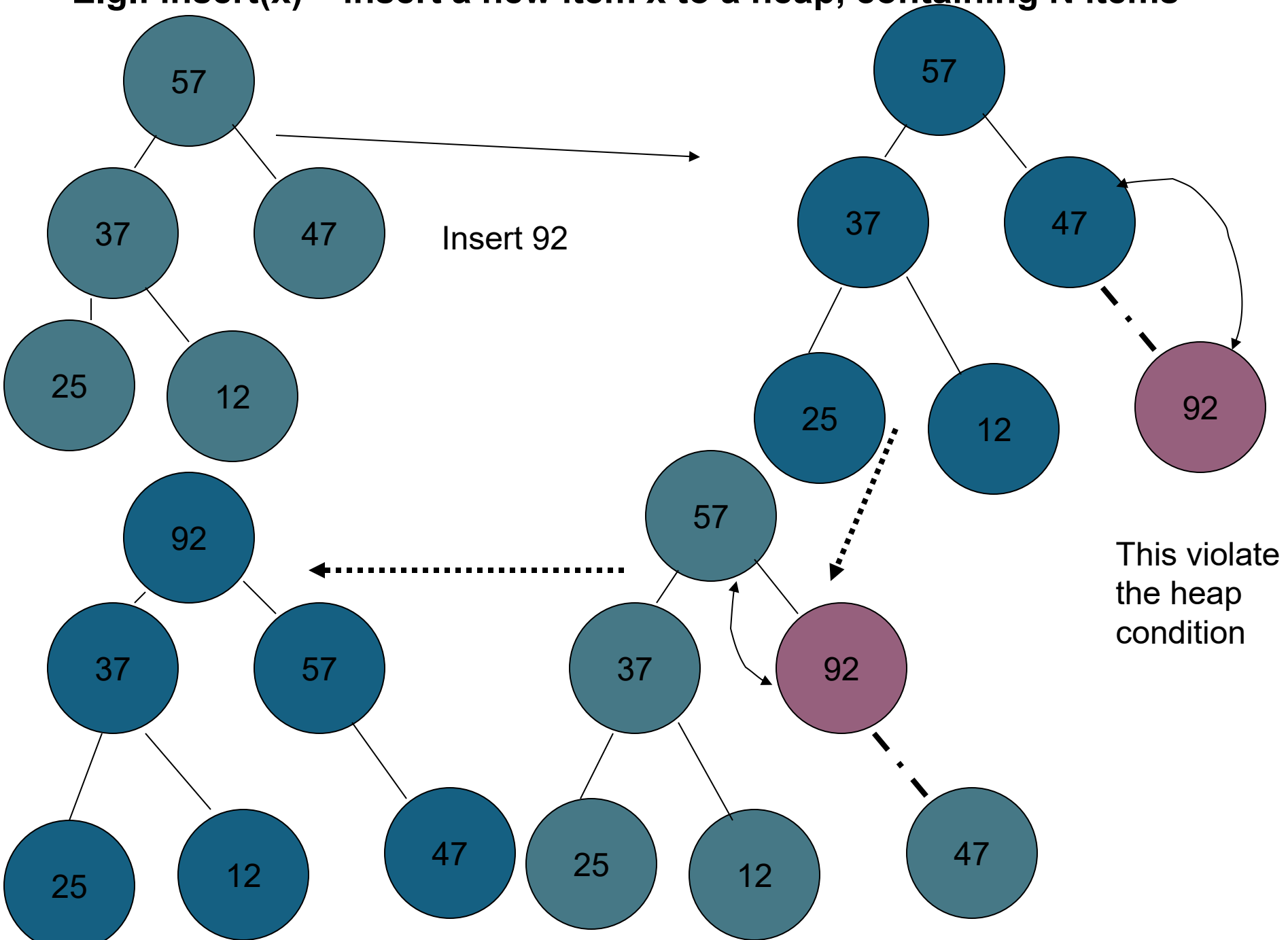


The tree may be easily constructed from the array by noticing that the left child of the k as 2k and right child 2k+1.

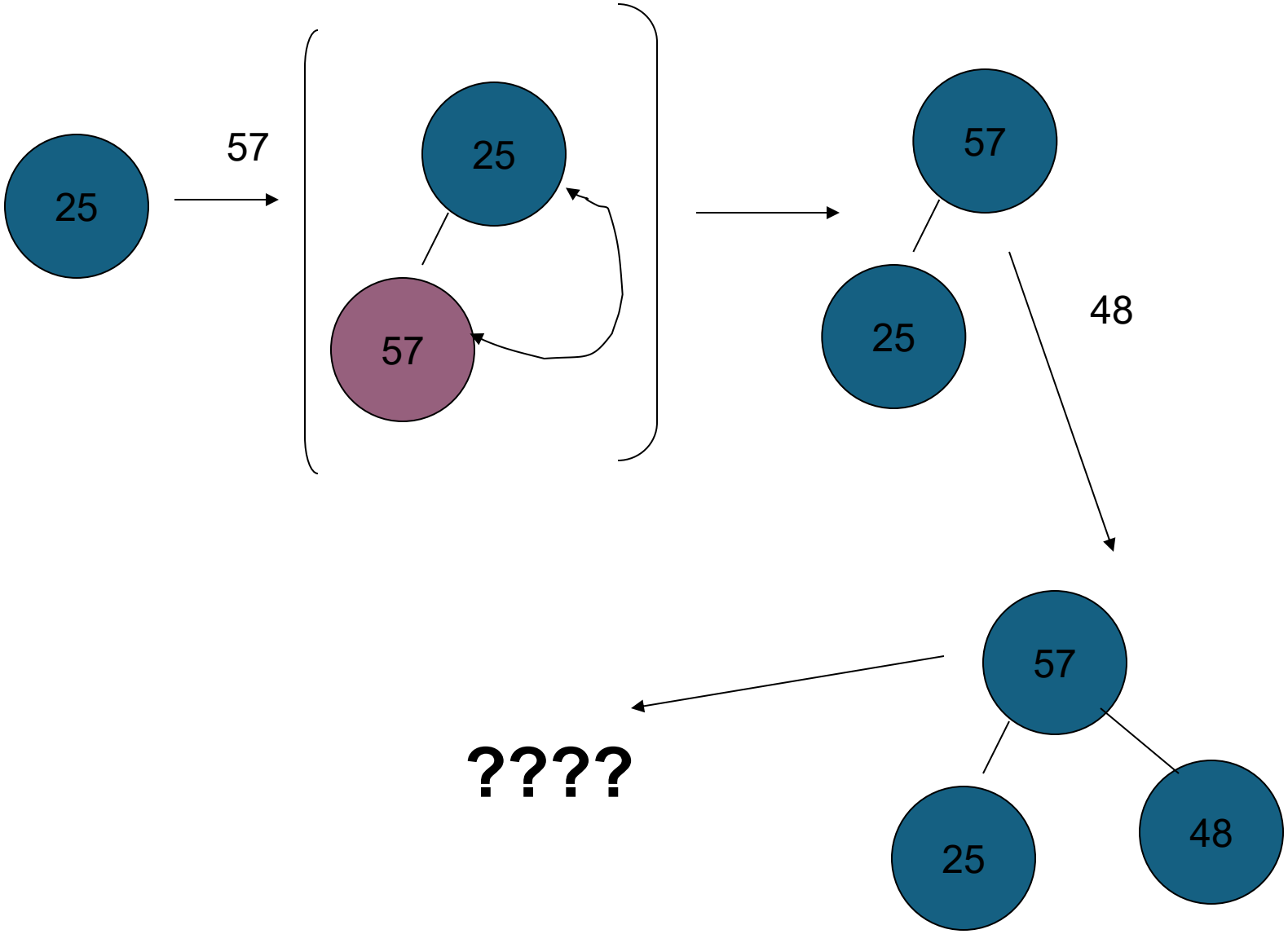
Algorithms on Heaps

- The algorithms on heap works by first making a simple structural modification which could violate the heap condition, the traveling through the heap modifying it to ensure that the heap condition is satisfied everywhere.
- Some of the algorithm travel through the heap from bottom to top, others top to bottom.

E.g.: insert(x) – insert a new item x to a heap, containing N items



Creating a Heap of size 8. The original file is 25,57,48,37,12,92,86,33

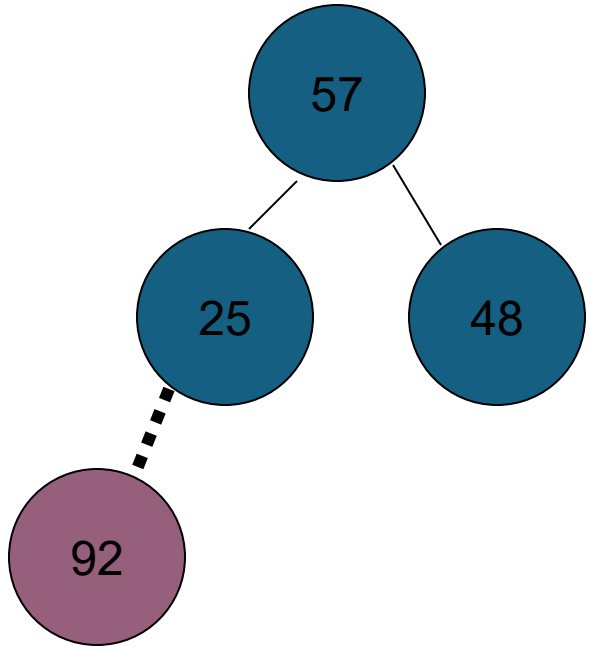


- We want to build and maintain a data structure containing records with numerical keys (priorities) and supporting some of the following operations.
- Construct a priority queue from given N items
- Insert a new item (with a priority)
- Replace the largest item with a new item.(unless new item is larger)
- Change the priority of an item.

- To be able build a heap, it is necessary, first implement the insert operation. This operation will increase the size of the heap by one. N must be incremented. Then the record to be inserted is put into A[N] , but this may violate the heap property (ie. New node is grater than its parent) then the violation can be fixed by exchanging new node with its parent .

- The code for this method is straight forward.
- Insert : add a new item to a[n], then upheap[n] fix the heap condition violation at N.

Pseudo code for insertion



Insert (X)

N++

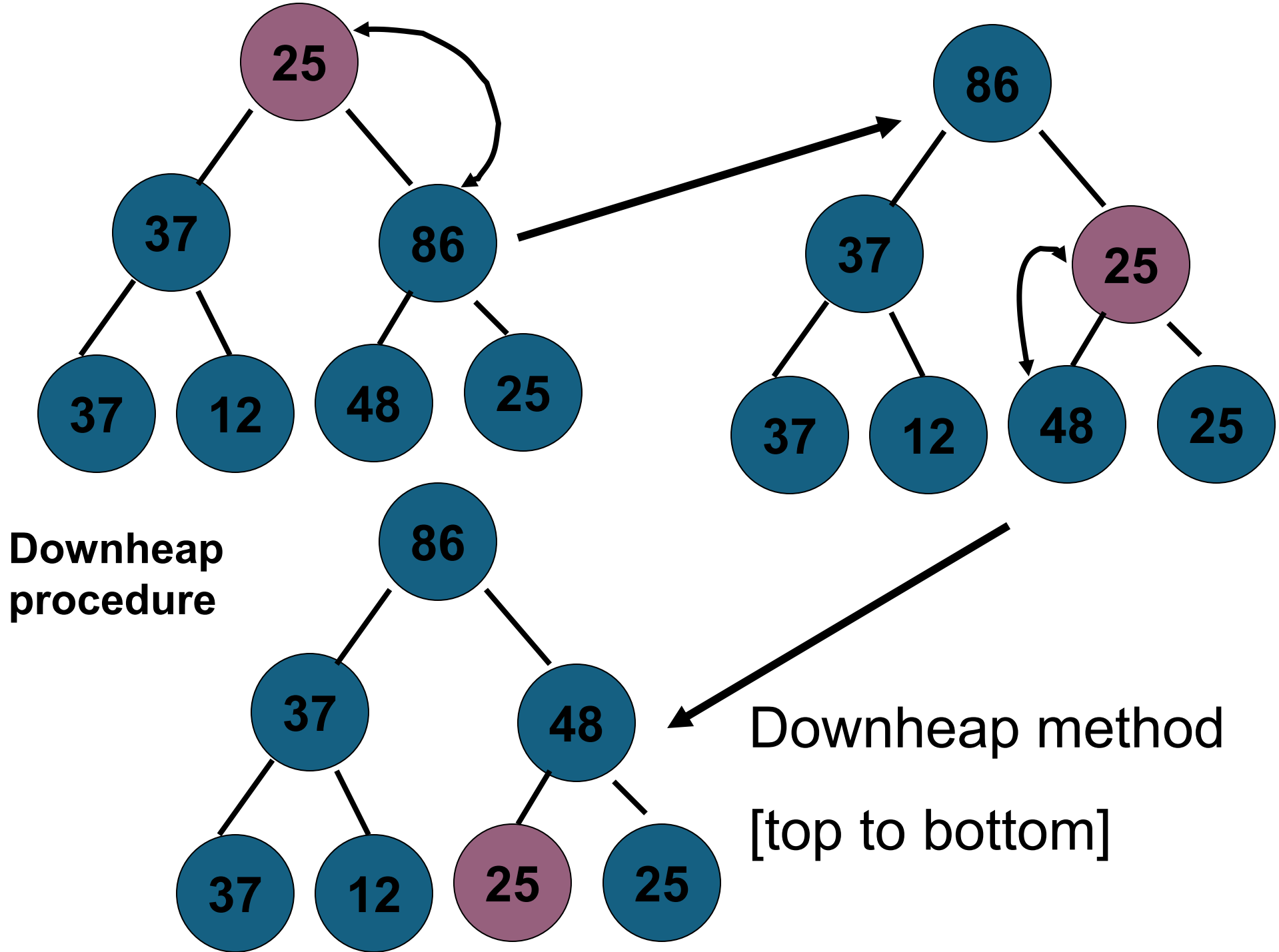
A[N]=x

Upheap{N]

Upheap procedure ??

Downheap Method

- If the element at node k is smaller than its children, that element is moved down the heap without violating the heap property at any nodes touched.


25
37
86
37
12
48
25
86
37
25
37
12
48
25
Downheap
procedure
86
37
48
37
12
25
25
Downheap method
[top to bottom]

Remove the largest item-pseudo code

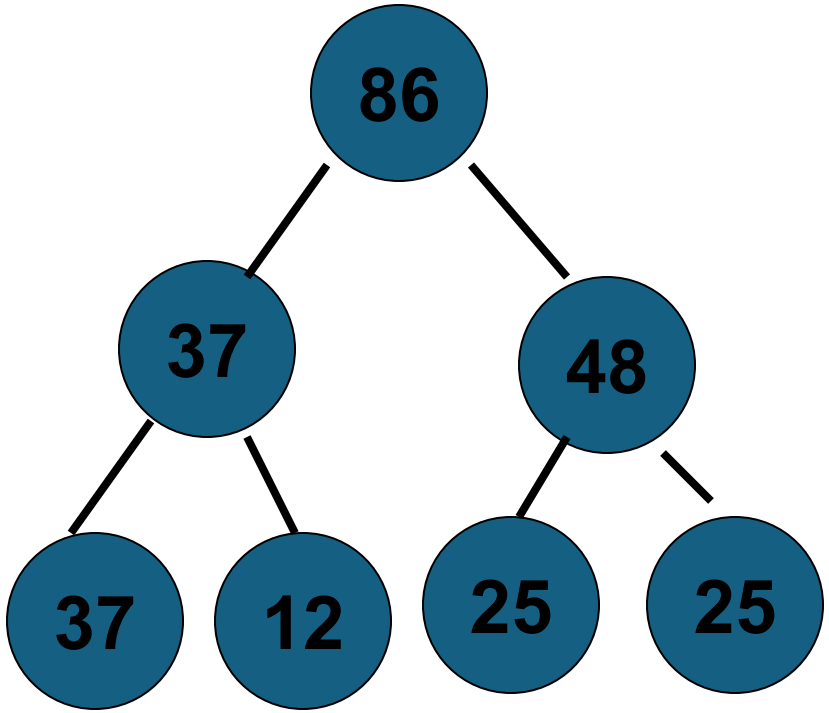
Remove the highest priority element ( ) ;

return = A[0];

A[0] = A[N-1];

N--;

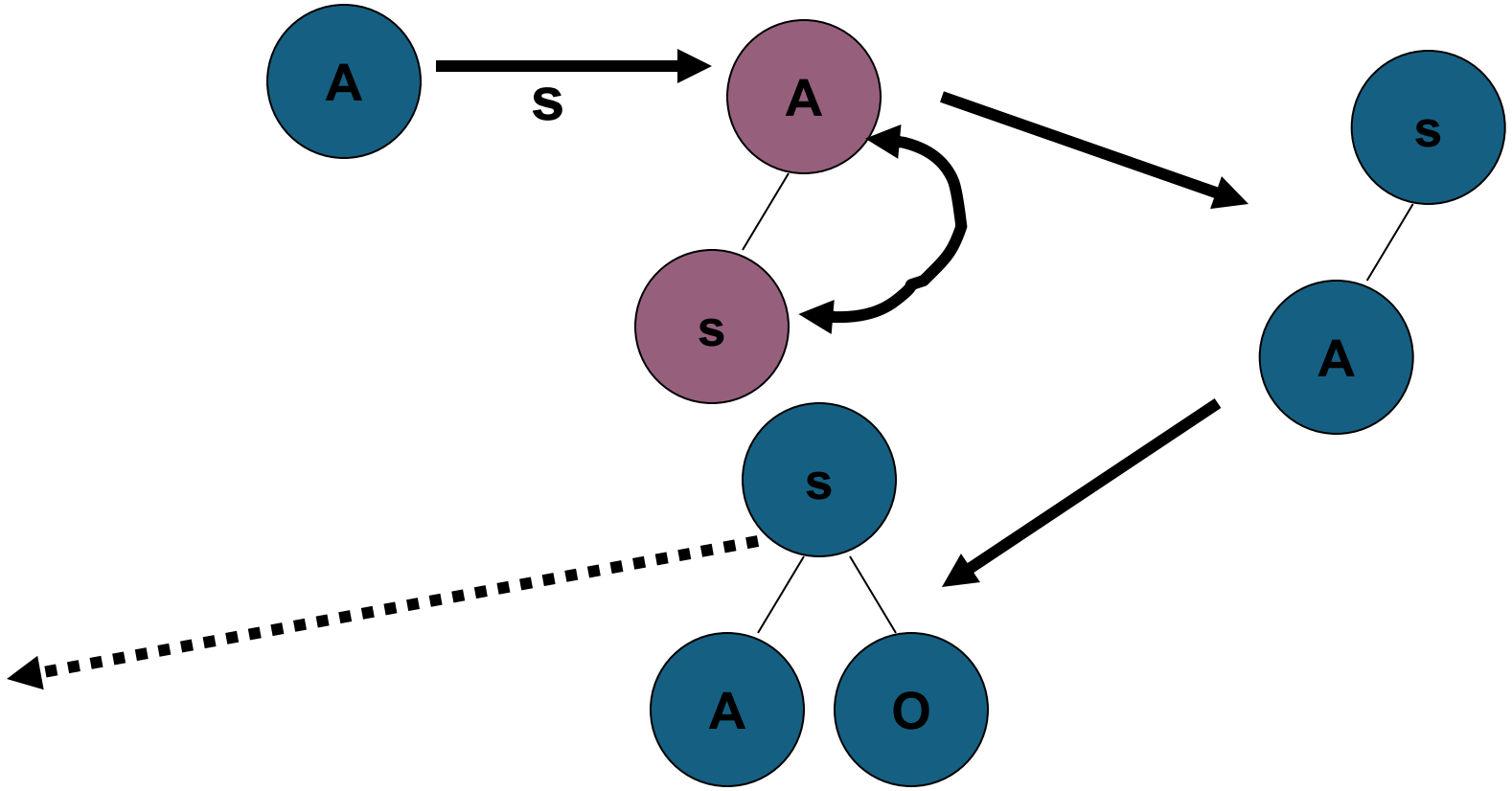
Upheap[N];



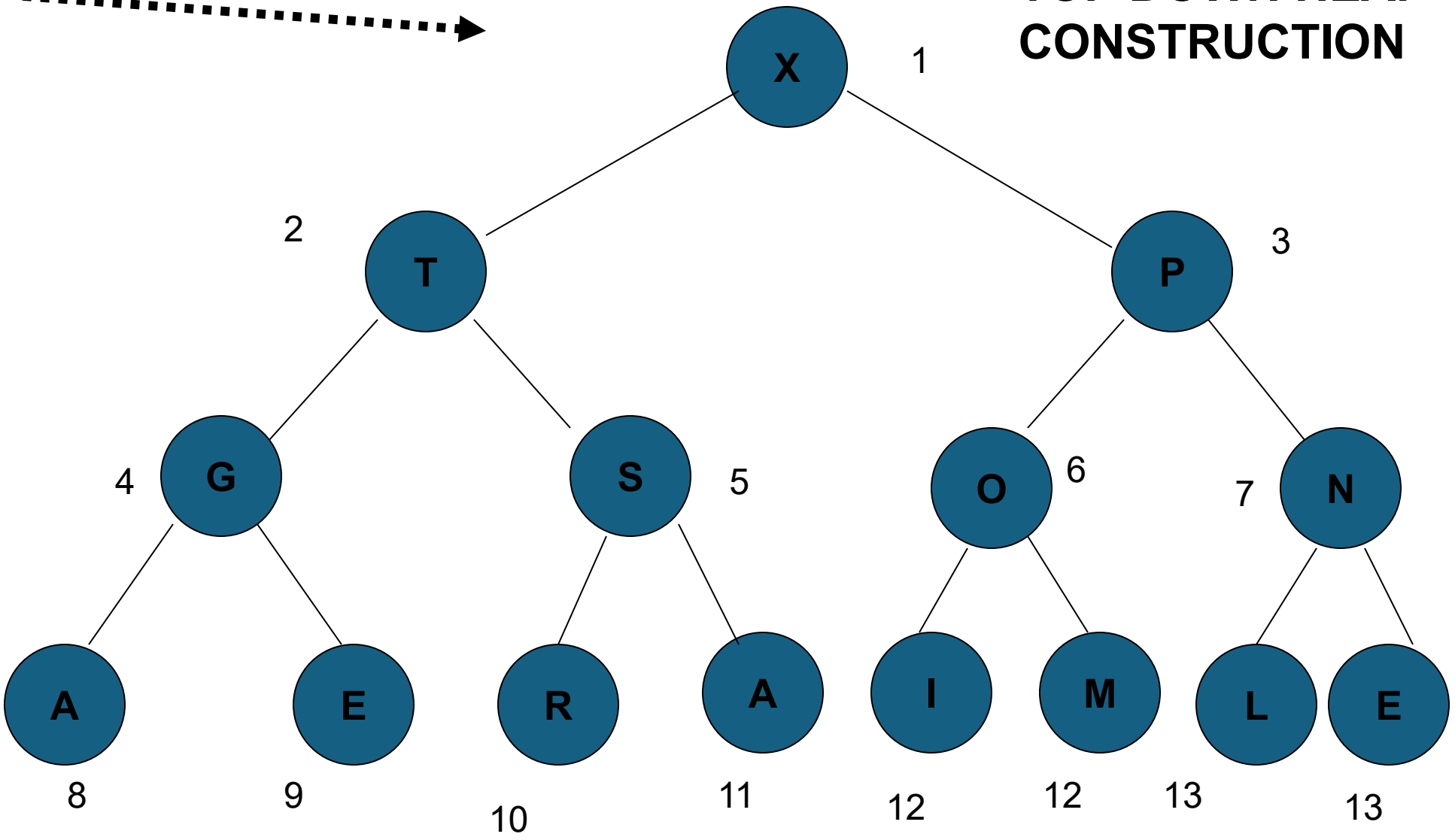
The return value is set from A[0] and then the element from A[N-1] is put into A[0] and the size of the heap is decremented, call down heap procedure to fix up the heap condition everywhere.

Heap Sort

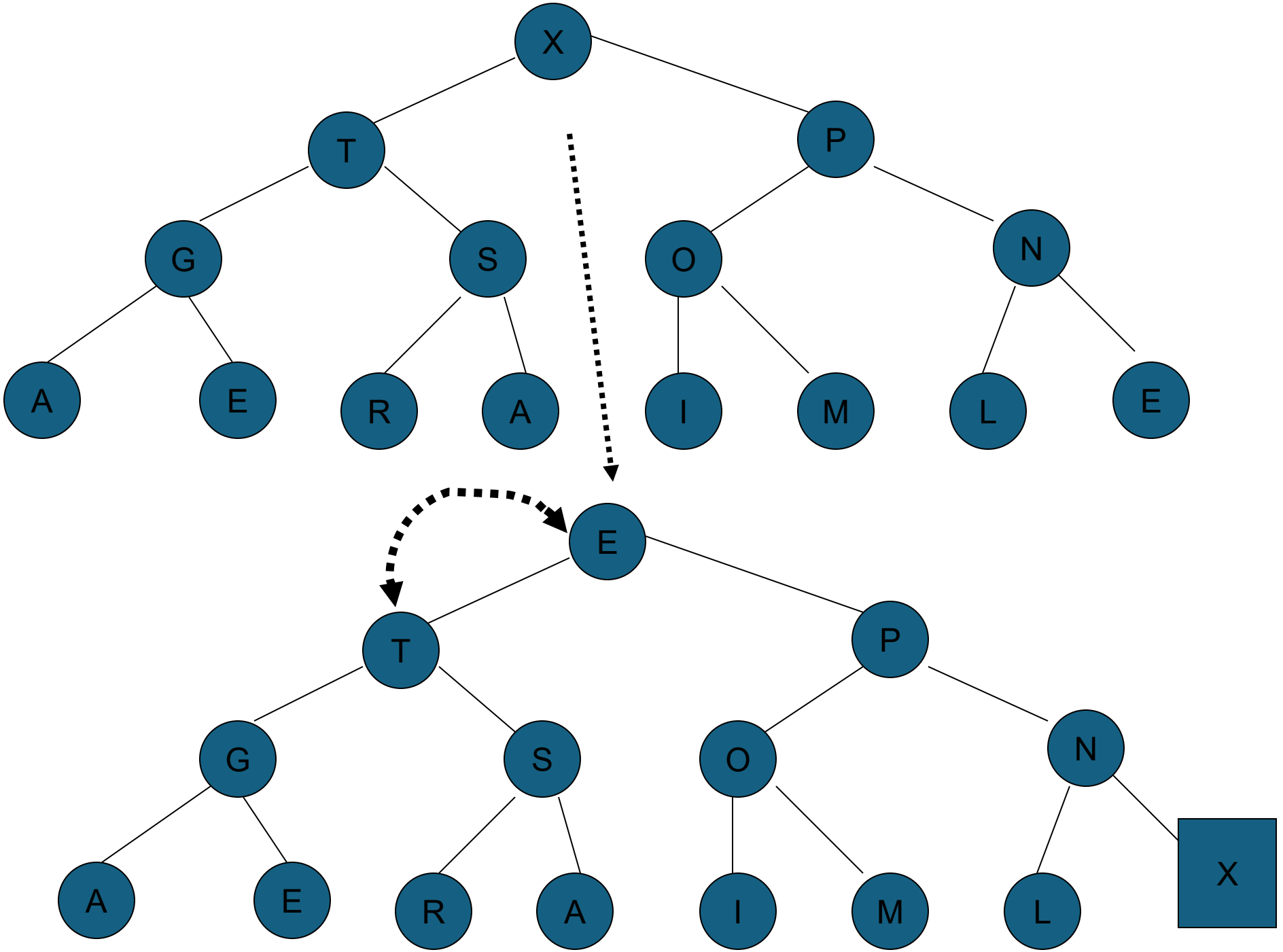
Eg : A SORTING EXAMPLE

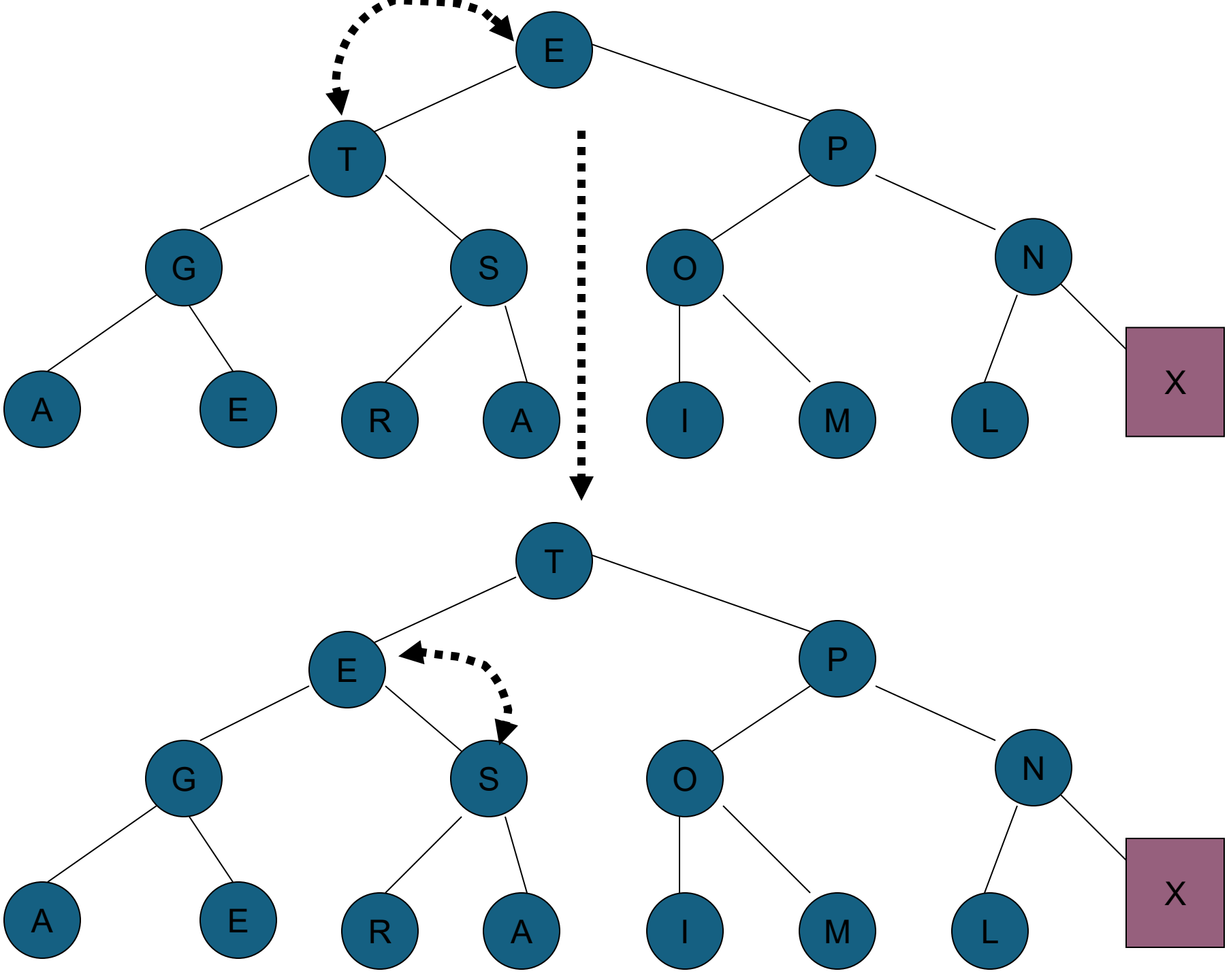


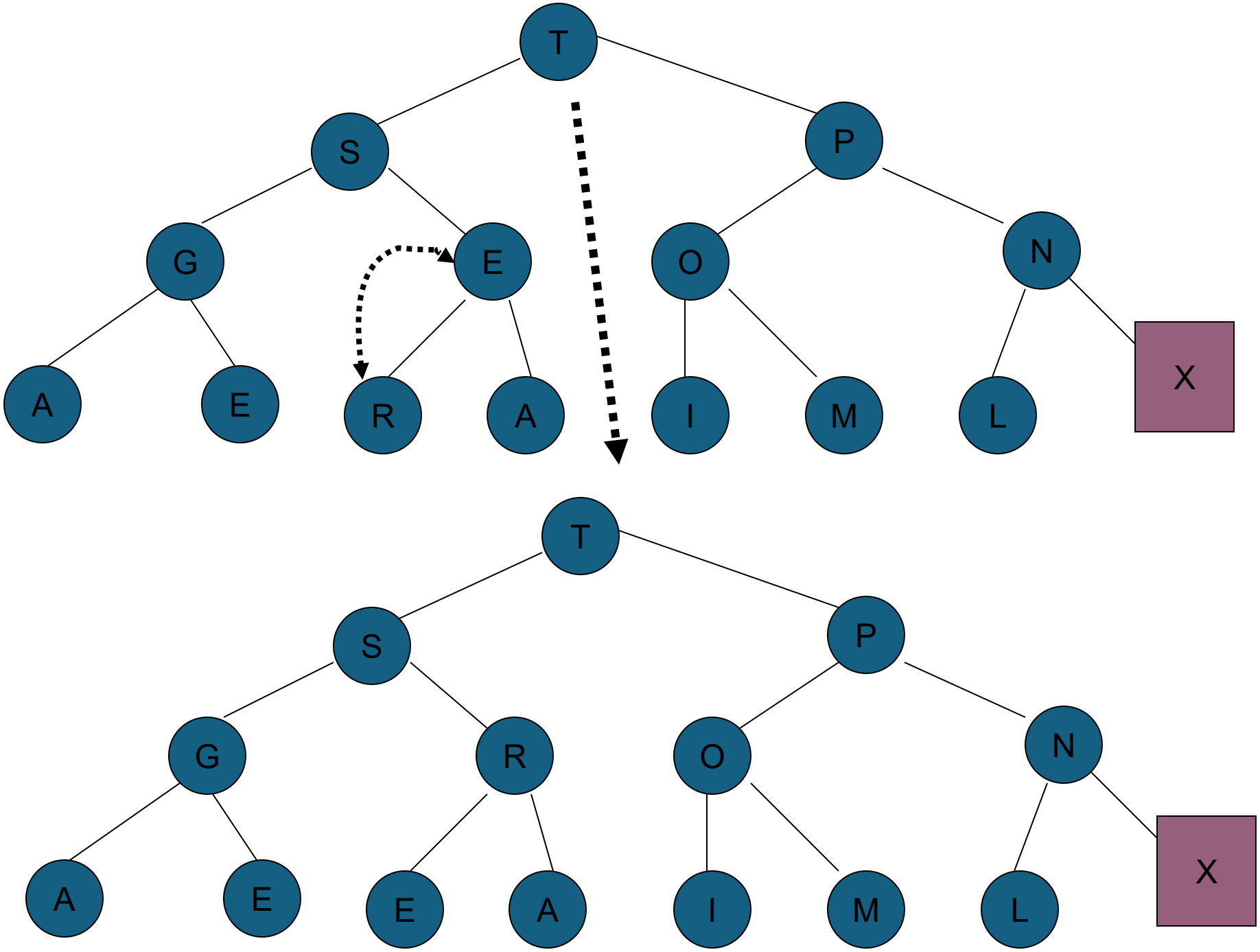
TOP DOWN HEAP CONSTRUCTION

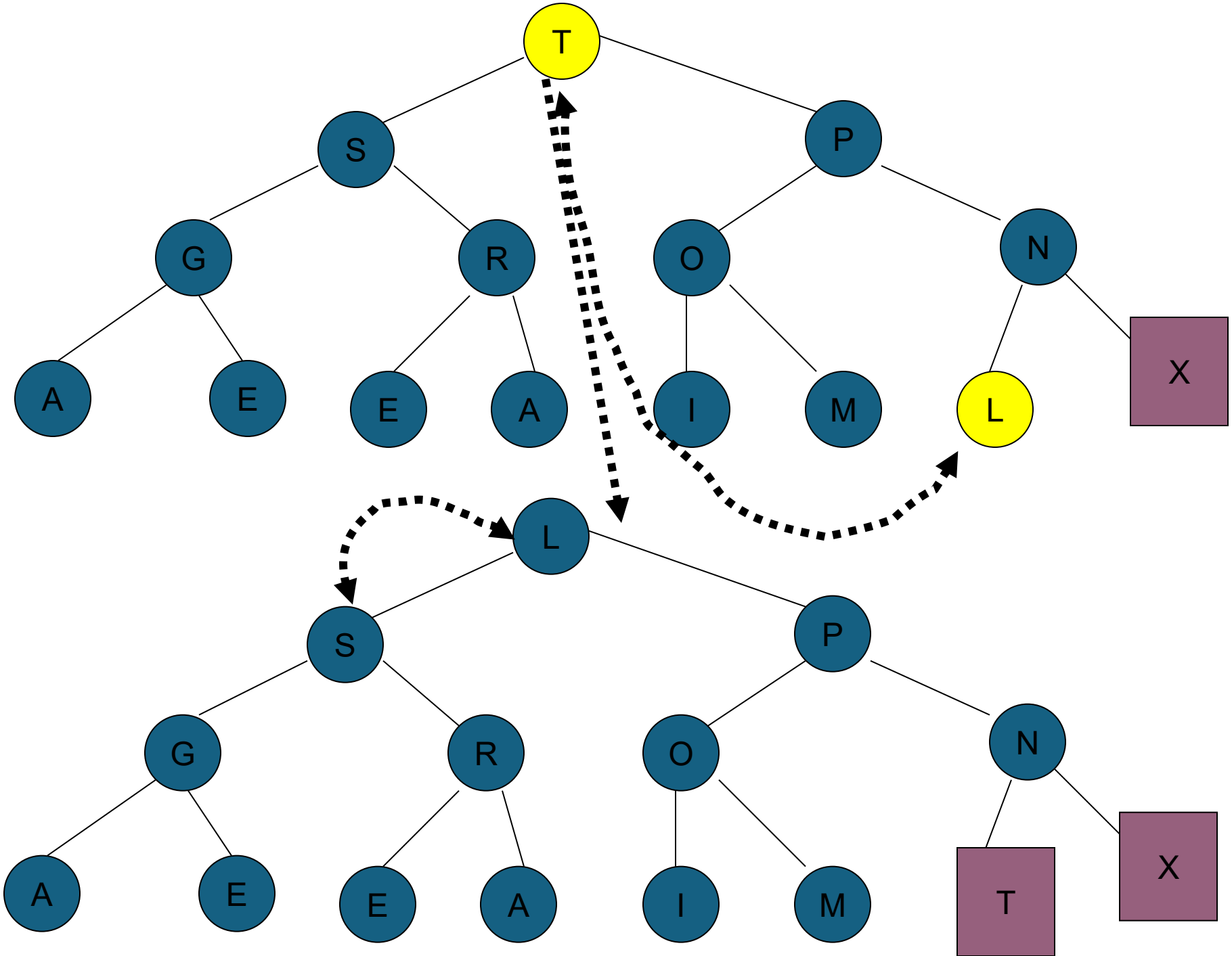


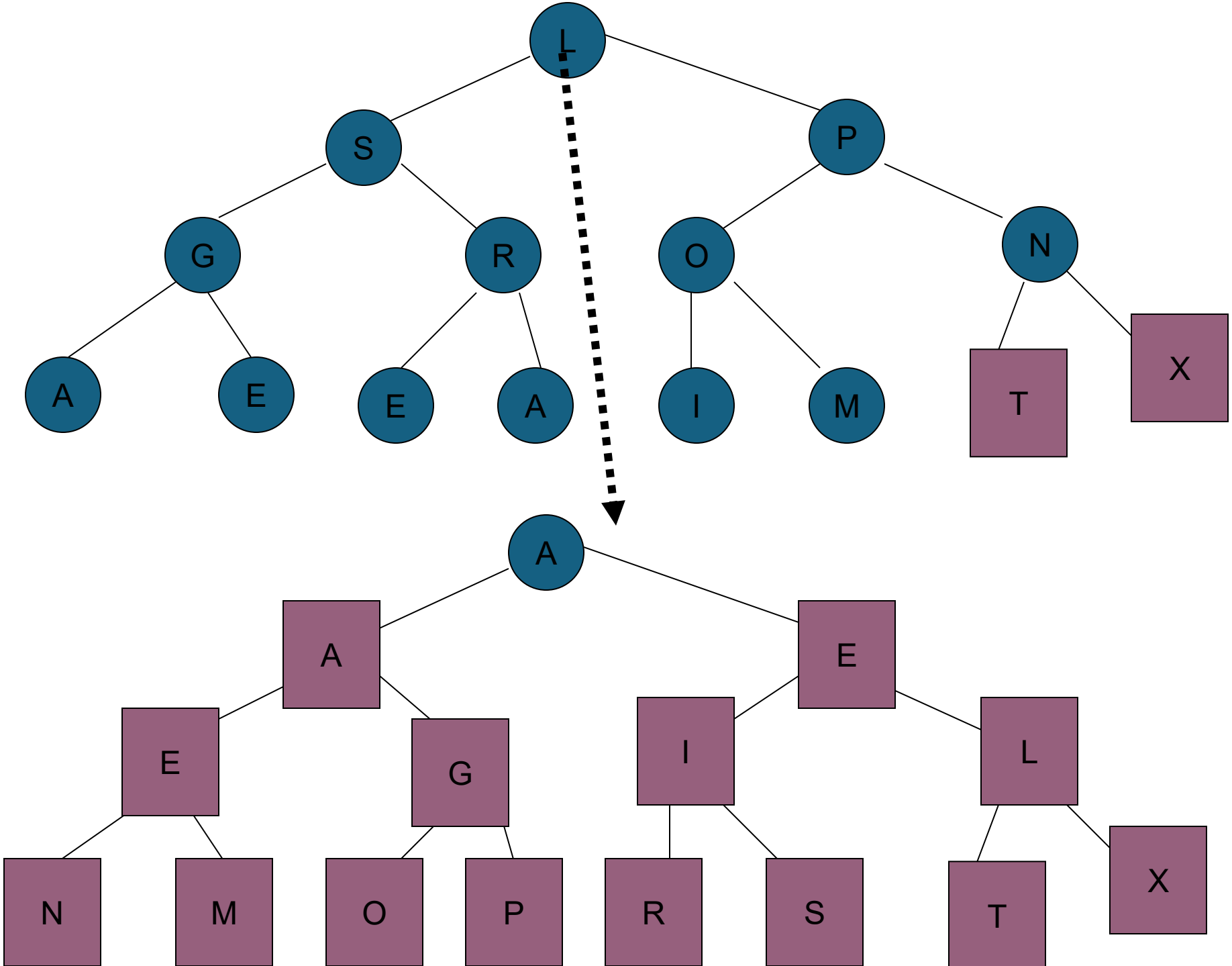
Above figure shows the heaps constructed as the keys A SORTING EXAMPLE are inserted in that order into an initially empty heap.


X
T
P
G
S
O
N
A
E
R
A
I
M
L
E
E
T
P
G
S
O
N
A
E
R
A
I
M
L
X


E
T
P
G
S
O
N
A
E
R
A
I
M
L
X
T
E
P
G
S
O
N
A
E
R
A
I
M
L
X


T
S
P
G
E
O
N
A
E
R
A
I
M
L
X
T
S
P
G
R
O
N
A
E
E
A
I
M
L
X


T
S
P
G
R
O
N
A
E
E
A
I
M
L
X
L
S
P
G
R
O
N
A
E
E
A
I
M
T
X


L
S
P
G
R
O
N
A
E
E
A
I
M
T
X
A
A
E
E
G
I
L
N
M
O
P
R
S
T
X

- 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
- A[I] A A E E G I L N M O P R S T X

- We were already noted that remove can be implemented by exchanging the first and last element and decrementing n by one and calling downheap from one.